UIPATH HACKWEEK · CODED AGENT

# Amplify — how it works

When a feature ships, Amplify turns it into sales-ready enablement — a one-pager and a video — grounded in the actual pull request, on-brand, and dropped straight into Data Service. This is the internal walk-through of every part.

**Trigger** Jira label `Amplify`

**Brain** Claude Opus 5

**Runtime** UiPath coded agent (LangGraph)

**Output** `EnablementAsset` + Storage Bucket

## 01 The shape of it

Four phases, one graph. Capture the context while it's fresh, let two directors author the story, produce the assets, and gate everything before anything is stored.

PHASE 1

### Capture

A Jira label fires the agent; it pulls the linked PRs and reads the real diff.

PHASE 2

### Author

The content director writes the story; the camera director decides where the eye goes.

PHASE 3

### Produce

Dynamic on-brand assets, voice, music, captions — rendered to a one-pager and a video.

PHASE 4

### Govern

QA and claim-check gates, then stored to Data Service as *unreviewed* — capture, not publish.

## 02 The trigger

Enablement fails when someone reconstructs context weeks later. So the trigger sits at the moment of maximum context: the ticket.

JIRA LABEL

**Amplify**

Add the label `Amplify` to a Jira issue and an Integration Service trigger starts the coded agent with the issue key. The agent reads the ticket, pulls the **linked pull-request URLs** from it, and hands them to ingest. No form, no copy-paste — labelling the ticket is the whole action.

`Jira issue + label Amplify` → `IS trigger → start agent` → `read issue → PR links` → `ingest`

## 03 Inside the graph

The agent is a LangGraph `StateGraph`: `ingest → author → render → store`. Each node is small and single-purpose; state flows between them. Here is what each one really does.

1

### Ingest & route

Resolve the ticket to a concrete pull request, fetch its real content, and decide which kind of asset to make.

> From the Jira issue it extracts GitHub links and classifies each: a `/pull/<n>` or commit URL is a **feature**; a `/releases/tag/…` URL is a **release**. It then calls `api.github.com` for the PR title, body, diff, files and commits — authenticated by a GitHub PAT kept in an Orchestrator **Credential asset** (`AmplifyGitHubPat`), read at run time and never in code.
>
> It also captures the **single point of contact** — the PR assignee or author — so every asset knows who to ask. Only `api.github.com` is ever called, built from the parsed owner/repo, so there is no arbitrary-host fetch.

`httpx` `GitHub REST` `Orchestrator asset` `URL classifier`

2

## Content director CORE IP

Reason over the PR and write what sales actually needs to hear — not a feature dump.

> Claude **Opus 5**, called through the UiPath **AI Trust Layer** (LLM Gateway, via `UiPathChatAnthropicBedrock`), turns the diff into a structured plan: `feature_name, product, value_prop, persona, when_to_use, talking_points, objections, scenes[], youtube_metadata`. It writes the hook as a customer problem, the positioning ("use this when a customer says…"), and objection handling — the sales layer a generic product tour lacks.
>
> The hard rule: **ground every claim in the diff.** A hallucinated benefit shown to a customer is the failure that kills trust, so the plan may not assert anything the PR doesn't support.

`Claude Opus 5` `AI Trust Layer` `VideoPlan v3 / ReleasePlan` `grounded`

3

## Camera director

Make screen-capture-with-zoom feel deliberate instead of nauseating.

> Where a demo has real footage, the camera holds a full, everything-visible view by default and pushes in **only on interactions** — selecting an activity, running it, the result — then eases back out. Targeting is emitted as timed zoom keyframes (a region of interest per moment); the motion itself — easing, hold, dissolve between beats — is hand-tuned, because bad auto-zoom is worse than none.

`ROI keyframes` `FFmpeg zoom/pan` `cross-dissolves`

4

## Dynamic assets & design tokens

Generate the on-screen material, on-brand by construction.

> Each beat is a bespoke HTML/CSS scene with authored GSAP motion — title cards, code-defined diagrams (e.g. a file → streaming Base64 → file round-trip), callouts, kinetic captions. Every colour and font comes from the UiPath **design-token layer** (`brand/uipath-tokens.json` → CSS variables: Robotic Orange, Agentic Teal, Deep Blue, Poppins, Inter). Own the tokens well and everything downstream is on-brand for free.

`GSAP` `design tokens → CSS vars` `code-drawn diagrams`

5

## Audio

Voice, music and captions — the cheap, big perceived-quality lift.

> Voice-over is synthesised with **ElevenLabs** (its duration drives scene timing, so it runs early). A background music bed is chosen by the plan's `music_mood` from a curated library and ducked under the voice; SFX mark scene seams. Captions are transcribed with Whisper and burned onto the frame inside the safe band.

`ElevenLabs TTS` `music library + duck` `Whisper captions`

6

## QA & claim-check gates

Two gates: does it render clean, and is every spoken claim true?

> A structural + runtime lint checks on-palette colours (token vars only), deterministic motion, and content inside the frame. Then the **claim-check** re-reads every line of narration against the PR diff and commits and flags anything unsupported — metrics, named integrations, absolutes — at `high` or `medium` severity. Safe to publish means render-QA is clean **and** no high-severity claim survives.

`lint (palette / motion / frame)` `claim-check vs diff` `severity gate`

7

## Render

Turn the plan into files.

> The renderer drives headless Chrome across a **paused** GSAP timeline, seeking frame by frame, then muxes with FFmpeg to MP4 — deterministic, reproducible. One-pagers and digests are pure HTML strings, so they render inside the serverless agent itself. The video, which needs the browser toolchain, renders on the render worker (see below).

`HyperFrames` `headless Chrome` `FFmpeg → MP4` `HTML → PDF`

8

## Store

Assets to the bucket, metadata to Data Service — the agent's job ends here.

> Files upload to the **Storage Bucket** `amplify-assets` under a fixed `amplify/<slug>/` prefix; the entity stores durable `bucket://` references (not expiring links). A row is written to the **Data Service entity** `EnablementAsset` with all the metadata, and crucially `reviewStatus = false` — because a merge is a capture trigger, not a publish trigger.

`Storage Buckets` `Data Service entity` `bucket:// refs`

9

## Review & surface

A human approves before anything customer-facing ships.

> A UiPath App reads `EnablementAsset`, resolves each `bucket://` reference to a fresh download, and shows the video / one-pager for review. Flipping `reviewStatus` is the approval gate; only then does an asset move toward YouTube, the enablement hub, or Slack.

`UiPath App` `review gate` `publish targets`

## 04 Where it runs

Two runtimes, on purpose. The agent is serverless; the frame-render lives on a worker with a browser — because headless-Chrome rendering can't run inside a serverless sandbox.

SERVERLESS

### The coded agent

A LangGraph Python agent (`uipath` + `uipath-langchain`) running in UiPath's managed agent runtime. It ingests, calls the model through the Trust Layer, gates, and stores.

- Runs end-to-end with no machine to manage
- **Releases render here** — one-pager + digest are pure HTML
- Reads secrets from Orchestrator assets at run time

RENDER WORKER

### The video renderer

An unattended robot / VM with the browser toolchain — Node, headless Chrome, FFmpeg. The agent hands it the plan; it renders the MP4 and uploads to the same bucket.

- Holds the one thing a sandbox can't: a real browser
- Invoked by the agent, writes to `amplify-assets`
- Same pipeline, same tokens — identical quality

**Why split?** A one-pager is text, so it renders in the agent and the release path is fully automatic. A video is a browser seeking a timeline frame by frame — that needs Chrome, which the serverless runtime doesn't host. Putting the frame-render on a worker keeps the agent light and the video real.

## 05 What gets stored

One row per asset in the `EnablementAsset` Data Service entity. Field names are camelCase; URLs are durable `bucket://` references the App resolves on demand.

| FIELD | TYPE | WHAT IT HOLDS |
|---|---|---|
| `assetType` | `string` | feature · release |
| `title` | `string` | The feature / release name |
| `product` | `string` | Concise product line — StudioWeb, Verticals, Maestro… |
| `description` | `string` | One-line "what it's about" shown under the title |
| `spoc` | `string` | Single point of contact — PR author / assignee |
| `sourceRef` | `string` | The PR / release URL it was generated from |
| `videoUrl` | `string` | `bucket://` reference to the MP4 |
| `onepagerUrl` | `string` | `bucket://` reference to the one-pager |
| `digestUrl` | `string` | `bucket://` reference to the release digest |
| `qaStatus` | `string` | Render-QA result — passed · flags |
| `claimCheckStatus` | `string` | Claim-check result — reviewed · flags |
| `reviewStatus` | `boolean` | Human approval — **false** on generation |

## 06 Built on

Two directors are the only things built from scratch; the platform provides the rest and Amplify configures it.

**UiPath Agents SDK**
`uipath · uipath-langchain`

**LangGraph**
`StateGraph orchestration`

**Claude Opus 5**
`AI Trust Layer · Bedrock`

**Data Service**
`EnablementAsset entity`

**Storage Buckets**
`amplify-assets`

**Orchestrator assets**
`GitHub PAT (credential)`

**Integration Service**
`Jira · GitHub`

**HyperFrames**
`headless Chrome + FFmpeg`

**GSAP**
`authored scene motion`

**ElevenLabs · Whisper**
`voice · captions`

`Amplify · from merged to market, automatically.`

Merge is the capture trigger, not the publish trigger — the moment of maximum context, caught before it evaporates.